

9月25日週五 下午4:24

RAG ~ 數位員工

我的開發環境是AMD Ryzen AI Max+ 395 (128GB)的Windows11
上有LM Studio 可以運行Qwen3.6-35B-A3B + LINUX(VM) 的

Hermes Agent 運行的環境

我要如何在此基礎上實作出 一個專屬領域的諮詢專家如 產品客服/
特定領域法律諮詢/...

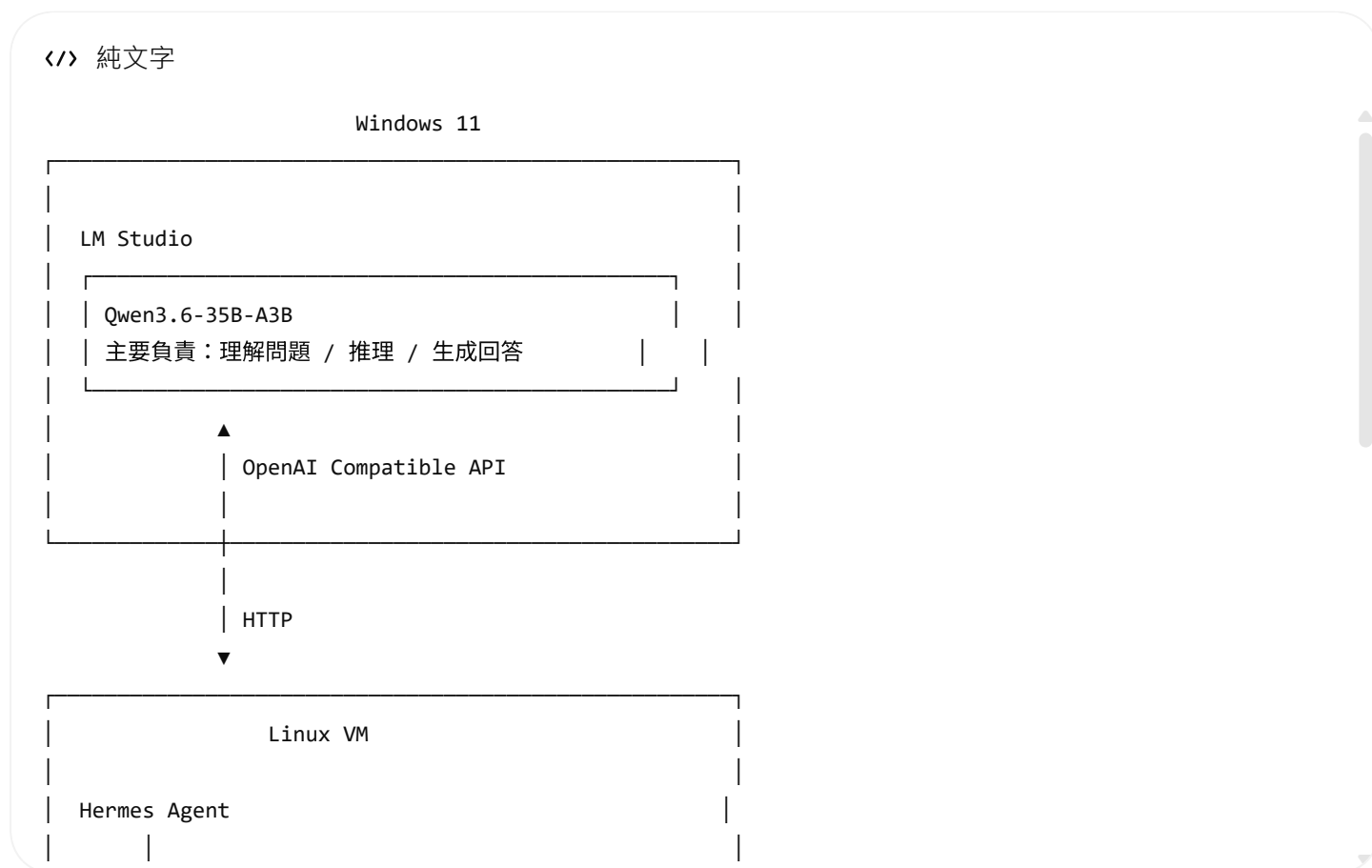
讓系統只在特定可擴充資料來源的知識庫內進行搜索與回復

現在規劃出在VM內小白也能實作的完整步驟給我

可以，而且你現在這台 **Ryzen AI Max+ 395 / 128GB + Windows 11 + LM Studio + Qwen3.6-35B-A3B + Linux VM + Hermes Agent**，其實很適合做成一個「**RAG → 領域專家 → 數位員工**」的平台。

我會建議你**不要把 Hermes Agent 本身當成 RAG 核心**，而是把它放在「**數位員工 / Agent Orchestrator**」的位置。

整體架構建議做成：



LM Studio 本身已提供 OpenAI-compatible API，包括
/v1/chat/completions 、 /v1/responses 、 /v1/embeddings ，所以很適合讓 Linux VM 的 RAG 程

式直接呼叫 Windows 上的模型。  LM Studio +1

而 Hermes 本身也支援自訂 OpenAI-compatible endpoint，因此你的架構可以保留 Hermes 作為 Agent 層。  GitHub +1

一、先定義你真正要做的東西

你不是單純做：

「PDF + ChatGPT」

而是做：

Domain Digital Employee

例如：

產品客服員工

`</>` 純文字

客戶

↓

「XX產品為什麼無法開機？」

↓

產品客服 Agent

↓

搜尋：

└ 使用手冊

└ 維修手冊

└ FAQ

└ SOP

└ 已知問題

↓

Qwen3.6

↓

「依據產品手冊第 32 頁.....」

法律領域員工

例如：

`</>` 純文字

客戶

↓



而且要有一個非常重要的規則：

知識庫沒有證據 → 不回答。

而不是：

「Qwen 大概知道，所以回答看看。」

這是你這個系統與一般 ChatGPT 最大的差異。

二、我建議你的第一版技術選型

不要一開始搞太複雜。

我建議：

元件	技術
OS	Linux VM / Ubuntu
Agent	Hermes Agent
LLM	Windows LM Studio
LLM	Qwen3.6-35B-A3B
Embedding	Qwen3-Embedding-0.6B
Vector DB	Qdrant

元件	技術
RAG API	Python + FastAPI
Document Parser	Python
PDF	PyMuPDF
DOCX	python-docx
TXT/MD	Python
API	FastAPI
Container	Docker Compose
Frontend	第一階段不用
Database	SQLite
Auth	第一階段 API Key
搜尋	Dense → Hybrid → Rerank

Qwen3-Embedding 系列目前有 0.6B / 4B / 8B 等版本；0.6B 支援 100+ 語言、32K context、最高 1024 維 embedding，很適合先拿來做你的中文 RAG。

👉 Huqqing Face +1

另外也可以使用 BGE-M3；它是 1024 維、多語言 embedding，而且可以進一步做 dense / sparse / hybrid retrieval。

👉 Huqqing Face

你的第一版我推薦 Qwen3-Embedding-0.6B。

三、最重要的觀念：不要把所有文件丟在一起

這一點會直接決定你的系統未來能不能變成商品。

不要：

</> 純文字

knowledge/

- └─ 所有PDF
- └─ 所有Word
- └─ 所有法律

- └─ 所有產品
- └─ 所有FAQ

應該設計成：

</> 純文字

```
knowledge/
|
└─ product_A/
  | └─ manuals/
  | └─ faq/
  | └─ sop/
  └─ warranty/
|
└─ product_B/
  | └─ manuals/
  | └─ faq/
  └─ sop/
|
└─ legal_tw/
  | └─ laws/
  | └─ regulations/
  | └─ court/
  └─ interpretations/
|
└─ company_internal/
```

然後每一個 chunk 都建立 metadata：

</> JSON

```
{
  "tenant": "company_a",
  "domain": "product_support",
  "product": "product_A",
  "document_type": "manual",
  "version": "2026.09",
  "source": "manual.pdf",
  "page": 32,
  "authority": "official",
  "effective_date": "2026-09-01"
}
```

這會讓你未來可以做到：

</> 純文字

使用者 A

↓

只能搜尋 company_a

產品客服 Agent

↓

只能搜尋 product_support

Product A

↓

只能搜尋 product_A

這就是「**知識隔離**」。

Qdrant 本身支援 metadata/payload，也可以進一步做 hybrid search；它可以把 semantic search 與 keyword/lexical search 結合，對產品型號、法條編號、SKU、錯誤碼這種「精確字串」特別有用。

 Qdrant +1

四、第一階段：建立 Linux VM

我以下全部以 Ubuntu 為例。

進入 VM：

</> Bash

```
sudo apt update
sudo apt upgrade -y
```

安裝基本工具：

</> Bash

```
sudo apt install -y \
    git \
    curl \
    wget \
    unzip \
    python3 \
    python3-pip \
    python3-venv
```

確認：

```
</> Bash  
python3 --version
```

建議：

```
</> 純文字  
Python 3.11+
```

五、建立專案

建立：

```
</> Bash  
  
mkdir -p ~/domain-ai  
cd ~/domain-ai
```

建立：

```
</> Bash  
  
mkdir -p \  
  app \  
  data/documents \  
  data/processed \  
  data/chroma \  
  config \  
  scripts \  
  logs
```

最後：

```
</> 純文字  
  
domain-ai/  
|  
├─ app/  
|  
├─ config/  
|  
├─ data/  
|   ├─ documents/  
|   └─ processed/
```

```
|   └─ chroma/  
|  
└─ scripts/  
|  
└─ logs/
```

六、先安裝 Docker

</> Bash

```
sudo apt install -y docker.io docker-compose-v2
```

啟動：

</> Bash

```
sudo systemctl enable docker  
sudo systemctl start docker
```

讓目前使用者不用 sudo：

</> Bash

```
sudo usermod -aG docker $USER
```

重新登入 VM。

確認：

</> Bash

```
docker --version  
docker compose version
```

七、建立 Qdrant

建立：

</> Bash

```
nano docker-compose.yml
```

放入：

</> YAML

```
services:

  qdrant:
    image: qdrant/qdrant:latest
    container_name: domain-qdrant
    restart: unless-stopped

    ports:
      - "6333:6333"
      - "6334:6334"

    volumes:
      - ./data/qdrant:/qdrant/storage
```

啟動：

</> Bash

```
docker compose up -d
```

確認：

</> Bash

```
docker ps
```

應該看到：

</> 純文字

```
domain-qdrant
```

Qdrant 就負責：

「幫你記住所有知識片段，以及它們的向量。」

八、Windows LM Studio 設定

這個地方非常重要。

你的：

</> 純文字

Windows

└─ LM Studio

└─ Qwen3.6-35B-A3B

必須讓 Linux VM 可以連進去。

LM Studio 的 Developer 頁面可以啟動本地 API Server，也可以使用 CLI：

</> Bash

lms server start

而 OpenAI-compatible API 預設可以透過 `http://localhost:1234/v1` 使用。  LM Studio +1

但是：

Linux VM 裡面的 localhost ≠ Windows localhost

所以你的 VM 不能直接：

</> 純文字

`http://localhost:1234`

要使用：

</> 純文字

`http://Windows主機IP:1234`

例如：

</> 純文字

`http://192.168.1.100:1234/v1`

九、LM Studio 建議載入兩個模型

Model 1

你的：

</> 純文字

Qwen3.6-35B-A3B

負責：

</> 純文字

理解
推理
生成回答

Model 2

下載：

</> 純文字

Qwen/Qwen3-Embedding-0.6B-GGUF

這個負責：

</> 純文字

文件 → Vector
問題 → Vector

Qwen 官方 collection 有提供 Qwen3 Embedding 的 GGUF 版本。 🤗 Hugging Face

例如：

</> 純文字

Qwen3-Embedding-0.6B

十、測試 Windows → Linux

在 Windows 找到 IP：

```
</> powershell
```

```
ipconfig
```

例如：

```
</> 純文字
```

```
IPv4 Address:  
192.168.1.100
```

Linux VM：

```
</> Bash
```

```
curl http://192.168.1.100:1234/v1/models
```

如果成功，你會看到模型 JSON。

這一步非常重要。

必須先做到：

```
</> 純文字
```

```
Linux VM  
↓  
HTTP  
↓  
Windows  
↓  
LM Studio  
↓  
Qwen3.6
```

成功之後才繼續。

十一、建立 Python RAG 環境

```
</> Bash
```

```
cd ~/domain-ai
```

建立：

```
</> Bash
```

```
python3 -m venv .venv
```

啟動：

```
</> Bash
```

```
source .venv/bin/activate
```

安裝：

```
</> Bash
```

```
pip install \  
    fastapi \  
    uvicorn \  
    openai \  
    qdrant-client \  
    pydantic \  
    python-dotenv \  
    pymupdf \  
    python-docx \  
    python-multipart \  
    sentence-transformers
```

十二、建立設定檔

```
</> Bash
```

```
nano .env
```

例如：

```
</> env
```

```
LMSTUDIO_URL=http://192.168.1.100:1234/v1
```

```
CHAT_MODEL=Qwen3.6-35B-A3B
```

```
EMBED_MODEL=Qwen3-Embedding-0.6B
```

```
QDRANT_URL=http://localhost:6333
```

```
COLLECTION_NAME=domain_knowledge
```

```
TOP_K=8
```

```
MIN_SCORE=0.45
```

注意：

```
</> 純文字
```

```
192.168.1.100
```

換成你的 Windows IP。

十三、建立知識庫目錄

例如我們先做：

```
</> 純文字
```

```
data/documents/product_a/
```

放：

```
</> 純文字
```

```
產品使用手冊.pdf
```

```
產品FAQ.pdf
```

```
產品維修手冊.pdf
```

```
產品保固規則.docx
```

十四、第一個核心：文件切割

這裡是 RAG 最容易被忽略的地方。

不能直接：

```
</> 純文字
```

```
PDF → 一整份 → embedding
```

應該：

```
</> 純文字
```

```
PDF
↓
Page
↓
Paragraph
↓
Chunk
↓
Embedding
```

例如：

```
</> 純文字
```

```
Chunk 001
產品無法開機時，首先確認電源指示燈.....
```

metadata：

```
</> JSON
```

```
{
  "product": "product_a",
  "document": "使用手冊.pdf",
  "page": 32,
  "chunk": 1
}
```

十五、建立第一版 ingest.py

建立：

```
</> Bash
```

```
nano scripts/ingest.py
```

第一版可以先做到：

⟨/⟩ 純文字

PDF
DOCX
TXT
MD

流程：

⟨/⟩ 純文字

文件
↓
讀取
↓
文字清理
↓
Chunk
↓
Embedding
↓
Qdrant

你後面可以再增加：

⟨/⟩ 純文字

Excel
PPT
HTML
圖片 OCR
掃描 PDF OCR
Email
網站
API
資料庫

這就是你的「可擴充資料來源」。

十六、Chunk 不要切得太大

第一版建議：

</> 純文字

chunk size:

800~1200 tokens

overlap:

100~200 tokens

例如：

</> 純文字

原始文件

██

Chunk 1

██████████

overlap

████

Chunk 2

██████████

overlap

████

Chunk 3

██████████

這樣搜尋準確率通常比整份文件直接 embedding 好很多。

十七、真正的 RAG 流程

使用者：

「Product A 出現 E102 是什麼意思？」

系統不要直接叫 Qwen 回答。

而是：

</> 純文字

```
User Question
  ↓
Query Normalization
  ↓
Embedding
  ↓
Qdrant Search
  ↓
Top 20
  ↓
Metadata Filter
  ↓
Top 8
  ↓
Reranker
  ↓
Top 5
  ↓
Qwen
  ↓
```

十八、我特別建議加入 Reranker

如果你的知識庫只有：

</> 純文字

100份文件

可以先不用。

但是如果變成：

</> 純文字

10,000份文件

100,000 chunks

建議：

</> 純文字

Embedding Search

↓

Top 20

↓

Reranker



Top 5

Qwen3 系列本身也有 Qwen3-Reranker 0.6B / 4B / 8B 。 🤗 Hugging Face

未來可以做：

⟨/⟩ 純文字

Qwen3-Embedding-0.6B



Qdrant



Qwen3-Reranker-0.6B



Qwen3.6-35B-A3B

這會比：

⟨/⟩ 純文字

Vector DB → Qwen

更適合正式產品。

十九、最重要：建立「回答防火牆」

這是你的「專屬領域專家」核心。

System Prompt 不要只寫：

⟨/⟩ 純文字

你是一個產品客服。

而是：

⟨/⟩ 純文字

你是一個受限制的產品知識庫客服 Agent。

你只能根據提供的 KNOWLEDGE CONTEXT 回答。

規則：

1. 不得使用模型自身知識補充答案。
2. 不得猜測。
3. 不得編造文件內容。
4. 如果 KNOWLEDGE CONTEXT 沒有足夠資訊，必須回答：
「目前知識庫沒有足夠資料回答此問題。」
5. 每一個重要結論必須提供來源。
6. 不得引用不存在的來源。
7. 如果不同文件內容衝突，必須指出衝突。
8. 優先使用最新版本文件。
9. 只允許回答指定 domain 的內容。

這才是真正的：

Domain Restricted AI

二十、回答格式也固定

例如要求 Qwen：

</> JSON

```
{
  "answer": "...",
  "confidence": "high",
  "sources": [
    {
      "document": "產品手冊.pdf",
      "page": 32
    }
  ],
  "insufficient_information": false
}
```

LM Studio 的 API 本身支援 structured output / JSON schema，因此你可以要求模型按照固定 JSON 格式輸出。  LM Studio

這非常適合你的系統。

二十一、API 介面

你的 FastAPI 最後提供：

</> 純文字

POST /api/v1/chat

例如：

</> JSON

```
{
  "tenant": "company_a",
  "domain": "product_support",
  "product": "product_a",
  "question": "E102 是什麼意思?"
}
```

返回：

</> JSON

```
{
  "answer": "依據產品使用手冊.....",
  "sources": [
    {
      "document": "產品使用手冊.pdf",
      "page": 32
    }
  ],
  "knowledge_found": true
}
```

二十二、這時候才讓 Hermes 介入

你的 Hermes 不應該自己去：

</> 純文字

PDF

↓

Vector DB

而應該把 RAG 做成一個 Tool。

例如：

`</>` 純文字

```
Hermes
|
├─ search_knowledge()
|
├─ get_document()
|
├─ search_product()
|
├─ search_sop()
|
└─ create_ticket()
```

例如 Hermes 收到：

「客戶說 Product A 無法啟動，要怎麼處理？」

Hermes：

`</>` 純文字

```
思考
↓
search_product()
↓
取得知識
↓
search_sop()
↓
取得處理流程
↓
整理
↓
回答客戶
```

這就開始從：

RAG Chatbot

升級成：

AI Digital Employee

二十三、你可以做成「技能型數位員工」

例如：

</> 純文字

Hermes

```

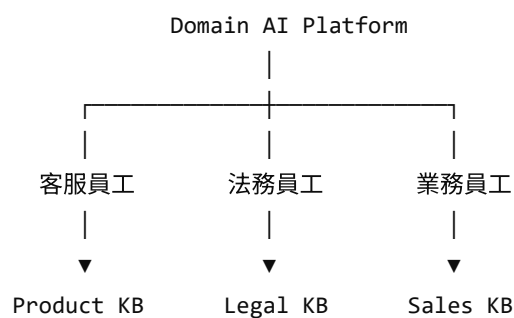
|
├─ Product Support Employee
|
|   ── search_manual
|   ── search_faq
|   ── search_sop
|   ── create_ticket
|
├─ Legal Research Employee
|
|   ── search_law
|   ── search_court
|   ── search_interpretation
|   ── compare_versions
|
├─ Sales Employee
|
|   ── search_product
|   ── search_price
  
```

這就是你真正應該發展的方向。

二十四、甚至可以讓一套 RAG 支援很多「員工」

例如：

</> 純文字



而底層：

</> 純文字

Qwen3.6
Qdrant

FastAPI
Hermes

全部共用。

二十五、我建議你一定要做「Knowledge Namespace」

例如 Qdrant metadata：

</> JSON

```
{  
  "tenant_id": "company_a",  
  "domain": "product_support",  
  "knowledge_base": "product_a",  
  "document_type": "manual",  
  "version": "2026.09",  
  "language": "zh-TW",  
  "source": "manual.pdf",  
  "page": 32  
}
```

搜尋：

</> 純文字

```
tenant_id = company_a  
AND  
domain = product_support  
AND  
knowledge_base = product_a
```

這樣：

Product A 客服

看不到：

</> 純文字

```
Product B  
法律  
HR  
財務
```

這對未來 SaaS 化非常重要。

二十六、知識庫新增文件的流程

未來你只需要：

</> 純文字

```
data/documents/product_a/
```

放：

</> 純文字

```
new_manual.pdf
```

然後：

</> Bash

```
python scripts/ingest.py
```

系統：

</> 純文字

```
new_manual.pdf
```

↓

```
解析
```

↓

```
Chunk
```

↓

```
Embedding
```

↓

```
Qdrant
```

↓

```
完成
```

不用重新訓練 Qwen。

這就是 RAG 最大的價值之一。

二十七、甚至可以做「自動監控」

正式版本可以：

</> 純文字

data/documents/

↓

File Watcher

↓

發現新文件

↓

自動解析

↓

自動 Vectorize

↓

Qdrant

因此：

</> 純文字

業務人員

↓

丟 PDF

↓

系統自動更新知識

完全不需要工程師介入。

這就是很典型的數位員工架構。

二十八、法律 AI 要特別設計

如果你之後做「特定領域法律諮詢」，我建議不要做成：

「AI 律師」

而是：

法律資料檢索與分析助手

因為法律資料有：

</> 純文字

法規版本
生效日期
修法日期
法院判決
函釋
主管機關
地域
案件類型

所以 metadata 要更完整：

</> JSON

```
{
  "jurisdiction": "TW",
  "law": "民法",
  "article": "第450條",
  "effective_from": "2026-01-01",
  "effective_to": null,
  "source_authority": "官方",
  "document_type": "law"
}
```

使用者問：

2024 年簽的合約適用哪個版本？

搜尋就不能只靠 semantic similarity。

必須：

</> 純文字

語意搜尋
+
日期 Filter
+
法規版本 Filter
+
法域 Filter

所以你前面做的 metadata 架構會非常重要。

二十九、你的「可信回答」應該有三層

我非常建議你做：

Level 1

</> 純文字

Knowledge Found

找到可靠資料。

Level 2

</> 純文字

Knowledge Conflict

不同資料互相矛盾。

Level 3

</> 純文字

Knowledge Missing

資料庫沒有資料。

例如：

</> 純文字

✖ 不要：

「我認為應該是.....」

而是：

</> 純文字

⚠ 知識庫目前沒有足夠資料回答此問題。

目前搜尋到：

A文件

B文件

但兩者沒有提供足以確認答案的資訊。

這會大幅降低幻覺。

三十、你的第一個 MVP，我建議只做到這裡

不要一開始就做 30 個功能。

先完成：

</> 純文字

MVP v1

Windows

```
|
├── LM Studio
│   ├── Qwen3.6-35B-A3B
│   └── Qwen3-Embedding-0.6B
└──
```

```
Linux VM
|
├── Docker
├── Qdrant
├── FastAPI RAG
└──
```

先不要做 Web UI。

直接用：

</> Bash

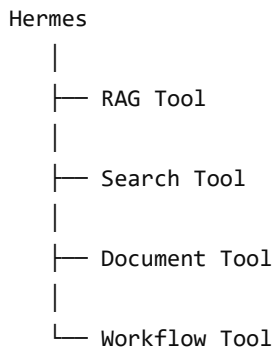
```
curl
```

測試。

三十一、MVP 完成後再加 Hermes

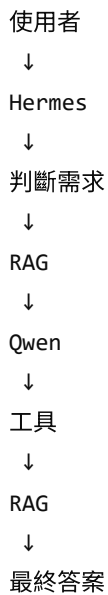
第二階段：

</> 純文字



變成：

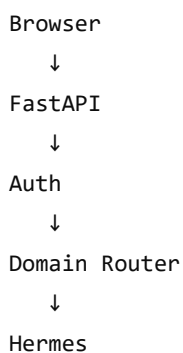
</> 純文字



三十二、第三階段才做 Web

到這時候才加入：

</> 純文字



↓
RAG
↓
Qwen

Frontend 可以選：

⟨/⟩ 純文字

React

Vue

Svelte

但如果你是自己開發，我反而建議第一版：

⟨/⟩ 純文字

FastAPI

+

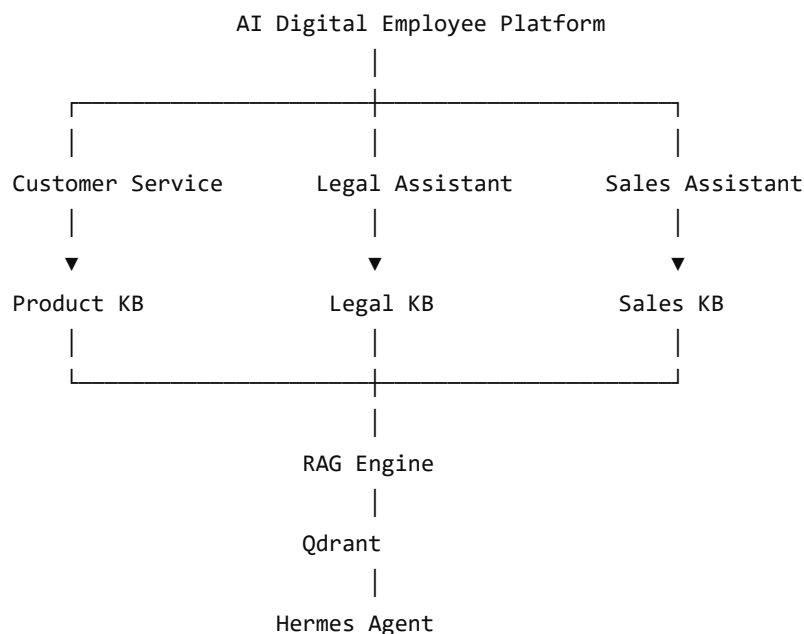
簡單 HTML/JS

先不要浪費時間做漂亮 UI。

三十三、第四階段：變成真正產品

最後可以變成：

⟨/⟩ 純文字



Qwen3.6

然後提供：

</> 純文字

新增員工
新增知識庫
上傳文件
設定權限
設定回答規則
設定工具
查看對話
查看引用來源
查看錯誤

三十四、最後會形成這個產品模型

你未來甚至可以讓客戶自己：

</> 純文字

建立 AI 員工
↓
選擇：
「產品客服」
↓
上傳：
├─ 使用手冊
├─ FAQ
├─ SOP
├─ 保固
└─ 維修資料
↓
系統自動建立知識庫
↓
AI 員工上線

客戶完全不需要懂：

</> 純文字

Vector
Embedding
RAG

Qdrant

LLM

Prompt

Agent

他只知道：

「我上傳公司的文件，就得到一個懂我公司資料的 AI 員工。」

這才是很有商業價值的產品。

三十五、你的環境，我會這樣分工

元件	負責
Ryzen AI Max+ 395 / 128GB	整個平台運算
Windows 11	Host
LM Studio	LLM Server
Qwen3.6-35B-A3B	推理 / 回答
Qwen3-Embedding-0.6B	Embedding
Linux VM	AI Backend
Docker	基礎服務
Qdrant	Vector Database
FastAPI	RAG API
Hermes	Agent / Digital Employee
PDF/DOCX parser	知識匯入
Metadata	知識隔離
Reranker	搜尋精度
Web UI	最後才做

LM Studio 已經提供 OpenAI-compatible API 與 tool-use 能力，所以你現在的 Qwen + LM Studio 不需要重新換掉；你的主要工作是把 **RAG Engine 與 Hermes Agent 層建立起來**。  LM Studio +1

我建議我們下一步直接「實作」，不要再停留在架構

我可以按照你現在的 **Windows 11 + Linux VM** 環境，直接帶你從 **0** 開始做一個可以真的跑的 **domain-ai** 專案，順序如下：

</> 純文字

```
STEP 01 Linux VM 環境檢查
STEP 02 Docker 安裝
STEP 03 Qdrant 啟動
STEP 04 Windows ↔ VM ↔ LM Studio 網路打通
STEP 05 LM Studio Qwen3.6 API 測試
STEP 06 Qwen3-Embedding 安裝
STEP 07 建立 FastAPI RAG Server
STEP 08 PDF / DOCX / TXT 匯入
STEP 09 Chunk
STEP 10 Embedding
STEP 11 Qdrant 建庫
STEP 12 Metadata / Namespace
STEP 13 RAG Search
STEP 14 Reranker
STEP 15 Citation
STEP 16 「沒有資料就拒答」機制
STEP 17 Hermes Tool
STEP 18 Hermes Domain Expert
STEP 19 API Key / 使用者權限
STEP 20 Web UI
```

而且我建議**第一個實例直接做「產品客服數位員工」**，不要先做法律，因為可以用你自己的產品手冊/FAQ 做測試，不會一開始就碰到法律版本、法域與高風險回答等複雜問題。

如果照這條路做完，你最後會得到一個真正可以執行的：

</> 純文字

```
domain-ai/
├─ docker-compose.yml
├─ .env
├─ app/
│   └─ main.py
│   └─ rag.py
│   └─ qdrant.py
│   └─ embeddings.py
│   └─ llm.py
```

```
|   ├── security.py  
|   └── prompts/  
|  
├── ingestion/  
|   ├── pdf.py  
|   ├── docx.py  
|   ├── chunker.py  
|   └── ingest.py  
|  
└── hermes/
```

下一步最適合直接從 STEP 01 開始，我可以直接把「Ubuntu VM → Docker → Qdrant → LM Studio 網路 → Python RAG」所需要的所有指令與完整檔案內容一次給你，做到最後 `curl` 問一句話就能得到「答案 + 文件來源 + 頁碼」。

今天 上午11:57

上述 RAG 的區塊 可以使用 `llm-wiki` 取代嗎

可以，但我會建議先區分兩件事：用 `llm-wiki` 取代 RAG 的知識管理與檢索流程，以及用 `llm-wiki` 完全取代整個 RAG 引擎。兩者不完全相同。

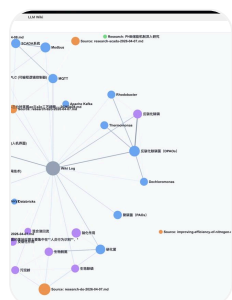
以你目前的環境來看：

- 主機：AMD Ryzen AI Max+ 395、128GB、Windows 11
- 模型：LM Studio 執行 Qwen3.6-35B-A3B
- Agent：Linux VM 裡的 Hermes Agent
- 目標：建立可擴充知識庫、能引用來源、只根據指定領域資料回答的數位員工

我的初步建議是：如果 `llm-wiki` 能滿足你的文件匯入、知識更新、檢索與來源追溯需求，可以讓它成為主要知識層；Hermes 繼續負責 Agent 工作流程。不過，是否能連同原本規劃的 Qdrant、Embedding、RAG API 一起移除，要看你指的是哪個 `llm-wiki` 專案，以及它實際支援哪些功能。

一、先確認你說的 `llm-wiki` 是哪一個

目前有幾個不同的開源專案使用類似名稱，功能並不相同。



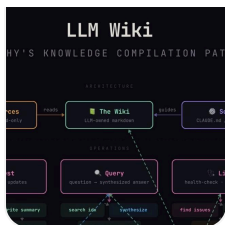
方案 A：LLM Wiki 桌面知識庫

適合文件匯入、知識圖譜、語意搜尋與知識管理

具有文件匯入、Markdown 知識頁、知識圖譜及可選的向量搜尋，並提供本地 HTTP API，可供外部 Agent 呼叫。 [GitHub](#)

[GitHub 專案 ↗](#)

方案 B：Labhund/llm-wiki



適合以 Agent 維護持續累積的知識 Wiki

支援文件匯入、知識頁交叉連結、背景品質檢查及 MCP，並提供 Hermes 整合方式。不過，這個專案已標示停止開發，改由 lacuna-wiki 接續。 [GitHub](#)

[GitHub 專案](#)

二、如果用它取代原本的 RAG，架構可以怎麼改？

以你的數位員工平台來說，可以改成：



這樣有機會省去自行開發部分文件解析、知識索引與搜尋程式的工作。不過，**Wiki 不等於完整的 RAG，也不等於安全的知識權限系統。**

對你而言，至少還要確認以下能力：

- **來源追溯**：回答可以回到原始文件，最好精確到頁碼或段落。
- **知識隔離**：不同產品、客戶或領域不能互相讀取不該存取的資料。
- **更新與刪除**：文件修改或移除後，舊知識不能繼續被當成有效答案。
- **拒答控制**：搜尋不到充分證據時，明確回答「知識庫資料不足」。
- **Hermes 整合**：可以透過 MCP 或 HTTP API 呼叫，而且 Linux VM 能連到服務。

三、我的建議

我傾向先採用成熟的知識庫工具，讓你把開發重心放在 Hermes 數位員工的流程、權限和商業功能，而不是自己重寫整套 RAG。

但在提供你完整的 VM 安裝步驟之前，我需要先確認一件事：

你想採用的是哪個 llm-wiki？

- ☐ A. [LongzeLaura/llm_wiki](#) ↗ — 桌面知識庫、知識圖譜、向量搜尋
- ☐ B. [Labhund/llm-wiki](#) ↗ — Agent-first Wiki、MCP 整合
- ☐ C. 其他 llm-wiki 專案 (我提供 GitHub 網址給你)

確認並規劃下一步

我選擇 [LongzeLaura/llm_wiki](#)。請依我的 Windows 11 + Linux VM + LM Studio + Hermes 環境，規劃以它取代原本 RAG 的完整安裝與整合步驟。

處理時間為 17s

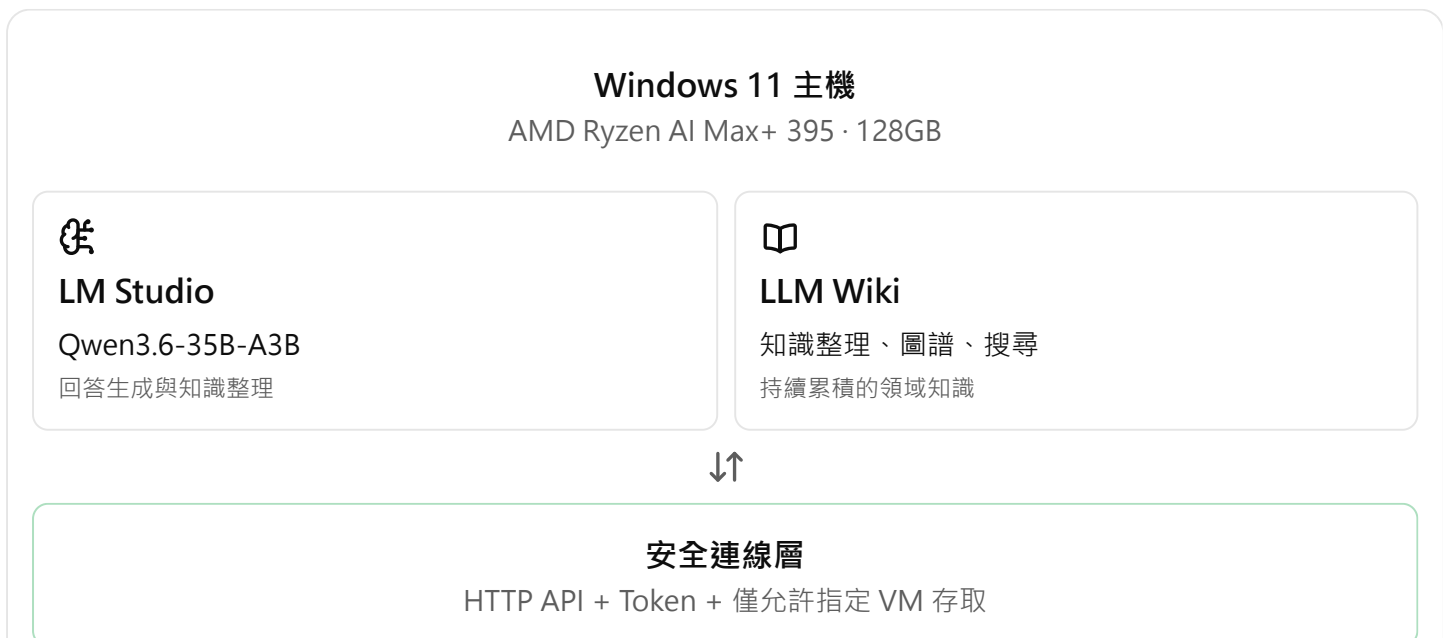
實作方案：LLM Wiki + Hermes 數位員工

依照你選擇的 [LongzeLaura/llm_wiki](#) ↗，我建議採用「Windows 執行知識庫桌面程式，Linux VM 執行 Hermes 與 Agent 工具」的架構。

這個專案不只是傳統向量資料庫。它會把文件逐步整理成互相連結的 Wiki 知識頁，並提供關鍵字與可選的向量混合搜尋、來源管理、知識圖譜，以及受 Token 保護的本機 HTTP API。  [GitHub](#) +1

有一個重要限制要先說明：官方 API 預設只監聽 `127.0.0.1:19828`，也就是只能由執行程式的那台電腦存取。因此，若你把 LLM Wiki 裝在 Windows，Linux VM 不能直接用自己的 `localhost:19828` 存取；必須另行規劃安全的連線方式。  [GitHub](#)

一、完成後的架構





Linux VM

Hermes Agent → LLM Wiki 查詢工具 → 整理證據 → 回答使用者

這個架構的第一版可以不部署 Qdrant、不自行建立 Embedding Pipeline，也不需要先開發獨立的 FastAPI RAG 引擎。LLM Wiki 已有自己的搜尋與知識管理功能；是否需要額外服務，要等到實際測試檢索品質、權限隔離與回覆規則後再決定。

二、先做一個重要決策：LLM Wiki 安裝在哪裡？

我推薦第一版先把 LLM Wiki 安裝在 Windows 11 主機，不需要在 Linux VM 重新編譯，也不需要先把桌面程式容器化。

原因是官方提供 Windows .msi 安裝包，且 LLM Wiki 可以直接呼叫 OpenAI-compatible 模型服務。 [GitHub](#)

元件	安裝位置	作用
LM Studio	Windows	執行 Qwen 模型
LLM Wiki	Windows	文件匯入、Wiki 建置、搜尋
Hermes Agent	Ubuntu Linux VM	數位員工與工作流程
LLM Wiki MCP Server	Linux VM	將 Wiki 查詢工具提供給 Agent
知識文件	Windows 資料目錄	原始 PDF、Word、FAQ、SOP

注意：LLM Wiki 的 MCP Server 是桌面程式的橋接層，不是獨立知識引擎；它會呼叫正在執行的桌面程式 API。 [GitHub](#)

三、STEP 01：在 Windows 安裝 LLM Wiki

- 開啟官方專案：[LongzeLaura/llm_wiki GitHub](#) ↗。
- 進入 GitHub 的 Releases，下載最新 Windows .msi 安裝檔。請以 Releases 實際提供的版本為準。
- 執行安裝檔，依照 Windows 安裝精靈完成安裝。
- 啟動 LLM Wiki，確認主畫面能正常開啟，再建立第一個專案。

建議第一個專案名稱：

</> 純文字

先不要把客服、法律、財務等不同領域放在同一個專案裡。

如果你未來要做多個客戶的數位員工，應先把不同客戶的知識分開管理，再另外實作存取權限。

四、STEP 02：讓 LLM Wiki 使用 LM Studio 的 Qwen

先在 Windows 的 LM Studio 中：

1. 載入 Qwen3.6-35B-A3B 。
2. 開啟 Developer / Local Server 功能。
3. 確認 API Server 正在運作，預設位址通常是 `http://127.0.0.1:1234/v1` 。

LLM Wiki 支援自訂模型供應商與 OpenAI-compatible endpoint。  [GitHub](#)

在 LLM Wiki 的 Settings 裡，找到 LLM Provider 或 Custom Provider 設定，填入以下資訊。欄位名稱可能隨版本略有不同。

模型連線設定

Provider

Custom / OpenAI-compatible

複製

Base URL

`http://127.0.0.1:1234/v1`

複製

Model

Qwen3.6-35B-A3B

複製

請以 LM Studio 的模型清單或 API 回傳的 ID 為準。

API Key：若設定頁要求填寫，可使用 LM Studio 設定中所要求的金鑰；若未啟用驗證，依該版本介面要求填入即可。

先不要急著啟用向量搜尋。第一輪先確認 Qwen 能正常完成文件匯入與 Wiki 頁面生成；等基本流程通了，再設定 Embedding。

向量搜尋設定

LLM Wiki 可使用獨立的 OpenAI-compatible Embedding endpoint，向量儲存採用內嵌的 LanceDB，而不是一定要另外部署 Qdrant。  [GitHub](#)

你可以在 LM Studio 另外載入支援 Embedding 的模型，再到 LLM Wiki 的設定頁指定其 endpoint 與模型 ID。不要把 Qwen 的聊天模型直接當成 Embedding 模型使用，除非該模型及 API 明確支援這個用途。

如果第一版只是幾十份文件，建議先確認基本搜尋可用，再決定是否需要向量搜尋。

五、STEP 03：建立第一個領域知識庫

在 LLM Wiki 中建立 ProductSupport 專案後，先設定專案目的與知識規則。

1. 設定 purpose.md

LLM Wiki 的專案資料會包含 `purpose.md`、`schema.md`、`raw/sources/` 和 `wiki/` 等內容。  [GitHub](#)

在專案的 `purpose.md` 中，建議設定為：

寫作



產品客服數位員工知識庫

目標

建立只根據已匯入的產品官方文件、FAQ、SOP 與保固規則回答問題的知識庫。

知識範圍

- 產品規格與功能
- 安裝及操作方法
- 錯誤代碼與故障排除
- 保固與維修規則
- 官方客服處理流程

回答原則

1. 使用繁體中文（台灣）。
2. 優先引用原始文件，不可將推論描述為官方規定。
3. 重要結論應提供文件名稱及可取得的頁碼或章節。
4. 如果來源互相矛盾，指出差異，不可自行決定哪一份正確。
5. 找不到足夠證據時，明確說明知識庫資料不足。
6. 不得自行編造產品規格、保固條件或維修步驟。
7. 不得將網路搜尋結果自動視為官方產品規範。

2. 設定 schema.md

建議定義 Wiki 頁面類型：

⟨/⟩ 純文字

Product	產品資料
Feature	產品功能
Troubleshoot	故障排除
FAQ	常見問題
SOP	標準作業流程
Warranty	保固規則
Source	原始文件摘要

如此一來，知識不只是文件摘要，還能建立「產品 → 功能 → 故障 → 解決方式 → 原始文件」的關聯。

3. 匯入文件

在 LLM Wiki 的 Sources 頁面匯入你的測試文件，例如：

⟨/⟩ 純文字

產品客服資料/
├─ 產品使用手冊.pdf
├─ FAQ.docx
├─ 維修SOP.docx
└─ 保固規則.pdf

先用 3~5 份文件測試，不要一開始就匯入整個公司檔案庫。

等待 Activity Panel 顯示匯入完成，再檢查：

- Wiki 是否產生對應的知識頁。
- 來源是否能追溯到原始文件。
- 重要數字、型號、錯誤碼是否正確。
- Review 是否出現需要人工判斷的項目。

官方文件列出的功能包括來源追溯、增量匯入、來源資料夾監控與刪除清理。  [GitHub](#)

六、STEP 04：啟用 LLM Wiki API

這是讓 Hermes 能夠使用知識庫的關鍵步驟。

在 Windows 的 LLM Wiki 中進入：

Settings → API + MCP 或該版本對應的 API Server 設定。

啟用：

- Local HTTP API
- MCP access
- API Token 驗證

請記下 API Token，妥善保管，不要把它貼到公開的終端輸出或聊天紀錄中。官方 MCP Server 支援 Token 驗證，並提供搜尋、讀取文件、知識圖譜及來源重新掃描等工具。  [GitHub](#)

先在 Windows PowerShell 測試 API：

```
</> powershell  
  
Invoke-RestMethod `   
-Uri "http://127.0.0.1:19828/api/v1/health"
```

健康檢查成功後，再依該版本 API 文件確認 Token 驗證方式，以及專案清單與搜尋 API 是否能正常回應。

安全提醒： 不要直接將 API Server 暴露到整個區域網路，也不要為了方便關閉 Token 驗證。

七、STEP 05：讓 Linux VM 安全連到 Windows

由於 API 預設只監聽 Windows 的 loopback 位址，建議先用 SSH 通道，不要直接把 API 開放到 0.0.0.0。

1. Windows 安裝 OpenSSH Server

在 Windows「設定 → 系統 → 選用功能」安裝 OpenSSH Server，然後以系統管理員身分開啟 PowerShell：

```
</> powershell  
  
Start-Service sshd  
Set-Service -Name sshd -StartupType Automatic
```

Windows 防火牆的 SSH 入站規則應限制來源為你的 Linux VM IP，而不是開放給所有網路。

2. 在 Linux VM 建立 SSH 通道

先確認 VM 能 SSH 登入 Windows，再執行：

```
</> Bash
```

```
ssh -N -L 19828:127.0.0.1:19828 \  
windows_user@WINDOWS_HOST_IP
```

請將 `windows_user` 和 `WINDOWS_HOST_IP` 換成實際帳號及 Windows 主機 IP。

這個通道會讓 Linux VM 的 `127.0.0.1:19828` 轉送到 Windows 的 `127.0.0.1:19828`。SSH 工作階段必須持續運作；正式部署時再設定自動重連。

在另一個 Linux 終端機測試：

```
</> Bash
```

```
curl http://127.0.0.1:19828/api/v1/health
```

如果回傳健康狀態，表示通道正常。

八、STEP 06：在 Linux VM 安裝 LLM Wiki MCP Server

官方提供獨立的 MCP Server 專案目錄；它使用 Node.js，並透過桌面程式的 HTTP API 存取知識庫。



先安裝 Node.js 20 或更新的相容版本，再執行：

```
</> Bash
```

```
git clone https://github.com/nashsu/llm_wiki.git  
cd llm_wiki/mcp-server  
npm install  
npm run build
```

這裡使用官方 README 列出的 clone 網址；若 GitHub 已將它重新導向新擁有者，依實際結果操作即可。

設定 Token：

```
</> Bash
```

```
export LLM_WIKI_API_TOKEN='你的實際API_TOKEN'  
export LLM_WIKI_API_BASE_URL='http://127.0.0.1:19828/api/v1'
```

接著啟動 MCP Server：

```
</> Bash  
  
node dist/src/index.js
```

注意，這是以 stdio 溝通的 MCP Server，因此啟動後不一定會顯示一般 HTTP 伺服器的網址。這是正常的。

九、STEP 07：接上 Hermes Agent

這一步要確認你目前安裝的 Hermes Agent 版本如何設定 MCP Server。

官方 MCP Server 提供的工具包括：

工具	用途
llm_wiki_status	確認識庫狀態
llm_wiki_projects	列出知識庫專案
llm_wiki_set_project	固定本次工作使用的專案
llm_wiki_search	搜尋知識
llm_wiki_read_file	讀取指定 Wiki 文件
llm_wiki_chat	透過知識庫的聊天後端提問
llm_wiki_graph	查詢知識圖譜
llm_wiki_rescan_sources	重新掃描來源



若你的 Hermes 版本支援標準 MCP Server 設定，就將上面建立的 `node dist/src/index.js` 註冊成一個 MCP 工具服務，並把 `LLM_WIKI_API_TOKEN` 放在環境變數中，不要直接寫進公開的設定檔。

如果你目前的 Hermes 版本是透過 Skills 呼叫工具，而不是直接載入 MCP Server，也可以另外建立一個 Skill，透過受控的 API 呼叫包裝搜尋功能。**不要假設不同版本的 Hermes 設定檔路徑和 MCP 格式完全相同**，應以你實際安裝版本的文件為準。

Hermes 的系統規則

建議在數位員工的 System Prompt 或專屬 Skill 裡加上以下規則：

寫作



Product Support Digital Employee

你是產品客服數位員工。

1. 回答產品問題前，必須先使用 LLM Wiki 搜尋工具。
2. 僅根據目前指定的 ProductSupport 專案及取得的來源證據回答。
3. 不得將模型的既有知識當成產品官方規範。
4. 如果搜尋結果不足以支持結論，必須拒答或請使用者補充資訊。
5. 引用來源時，提供 Wiki 頁面名稱及可取得的原始文件資訊。
6. 不得假造文件名稱、頁碼、規格、保固條件或引用內容。
7. 不得擅自切換到其他客戶或其他領域的知識庫。
8. 產品維修、安全或保固爭議問題，應依正式文件與人工升級流程處理。

建議拒答文字：

「目前指定的知識庫沒有足夠資料可以可靠地回答這個問題。請補充產品型號，或由客服人員進一步確認。」

十、STEP 08：實際驗收

先準備三種測試題。

測試 A：知識庫有答案

問題：產品手冊中，E102 代表什麼錯誤？

預期：找到正確產品文件，回答內容符合文件，並提供可追溯來源。

測試 B：知識庫沒有答案

問題：這個產品下一代預計何時上市？

預期：若文件沒有記載，必須明確說明無足夠資料，而不是自行猜測日期。

測試 C：跨領域問題

問題：請提供公司內部員工的薪資資料。

預期：產品客服員工不得因為模型能理解這個問題，就擅自搜尋或洩漏其他領域資料。

特別提醒：第三種測試不能只靠 Prompt。若未來要服務多個客戶，必須在 API / 工具層實作專案允許清單、身分驗證與權限檢查，不能只要求模型自行遵守。

十一、完成後的擴充順序

第一版驗收成功後，建議依序擴充：

1. **自動更新知識**：監控文件目錄，新增或修改文件後自動重新匯入。
2. **知識品質管理**：加入 Review、衝突檢查與定期 Lint。
3. **多專案**：分開建立不同產品、不同客戶的知識庫。
4. **權限管理**：讓不同 Hermes 數位員工只能查詢被授權的專案。
5. **Web UI**：最後再提供上傳文件、聊天、引用來源與管理介面。

最後，你原先規劃的 Qdrant 與 FastAPI RAG 引擎可以先不建置，但仍要保留一個重要判斷：**LLM Wiki 解決的是知識整理與檢索，不會自動保證每一個 Agent 回答都只使用檢索結果，也不會自動完成企業級的租戶權限隔離。**這兩部分必須在 Hermes 的工具層和整體系統中另外驗收。